

Kreacijski paterni

Factory pattern se koristi da se kreiranje objekata sakrije iza posebnih klasa, tako da se korisnik ne mora baviti direktnim stvaranjem objekata pomoću new operatora. Iskoristili smo ga za kreiranje različitih tipova korisnika, gdje KorisnikFactory i njegove izvedene klase poput AdministratorKorisnikFactory i MenadzerKorisnikFactory određuju koji tip korisnika će biti kreiran.

Builder pattern se koristi kada želimo postepeno kreirati kompleksan objekat, korak po korak. Mogao bi se implementirati kroz IzvjestajBuilder, koji omogućava da se izvještaj gradi postavljanjem parkinga, vremenskog perioda i statistike, a zatim se na kraju poziva metoda build() koja vraća kompletan izvještaj.

Prototype pattern omogućava kreiranje novih objekata kopiranjem već postojećeg objekta umjesto njegovog ponovnog pravljenja od nule. Primjenili smo ga na klasu Notifikacija, gdje se postojeća rezervacija može klonirati pomoću metode clone() i zatim izmijeniti, na primjer promjenom datuma ili drugih podataka.

Singleton pattern osigurava da postoji samo jedna instanca određene klase u cijelom sistemu i da se toj instanci pristupa globalno. Može se primjeniti za klasu Cjenovnik, gdje se preko metode getInstance() osigurava da uvijek postoji samo jedan cjenovnik koji koristi cijeli sistem.

Abstract Factory pattern se koristi za kreiranje porodica povezanih objekata bez eksplicitnog navođenja njihovih konkretnih klasa. Može se koristiti za različite tipove korisnika, gdje svaka factory klasa kreira odgovarajuću vrstu korisnika, kao što su administrator ili menadžer, zajedno sa njihovim specifičnim ponašanjima.